1) **Dado un arreglo de N enteros, cuyos valores van en decremento y luego incremento, encontrar el menor número en el arreglo.**

N= [2, 1, 2, 3, 4]; N = [8, 5, 4, 3, 4, 10]

Para este ejercicio la mejor opción es búsqueda binaria, ya que tiene una estructura ordenada de una forma específica (crece y luego decrece), lo que permite garantizar que el mínimo se encuentra en un subintervalo definido y que, al comparar el elemento medio con su vecino, se puede descartar la mitad del arreglo en cada iteración, asegurándonos de que el número más pequeño siempre va a estar en el rango considerado logrando una solución eficiente.

Complejidad

Línea	Costo	Tiempo
low=0	c_1	1
high=len(A)	c_2	1
while low<high:	c_3	$\log(n+1)$
mid=low+(high-low)//2	c_4	$\log n$
if A[mid]==k:	c_5	$\log n$
return mid	c_6	$1;\ A[mid] == k$
else:	c_7	1
if a[mid]>k:	c_8	$\log n$
High=mid	c_9	$\log n\,;\ A[mid] > k$
Else:	c_{10}	1
Low=mid+1	c_{11}	$\log n$

Teorema Maestro

$$T(n) = aT(n/b) + f(n)$$

Donde

$$a = 1;\ b = 2;\ f(n) = \Theta(1)$$

Calculo de n^log_b a

$$n^{\log_b} a = n^{\log_2} 1 = n^0 = 1$$

Resultado

Según el caso 2 cuando $f(n) = \Theta\left(n^{\log_b} a\right)$, entonces $T(n) = \Theta\left(n^{\log_b} a \log n\right)$ y sustituimos los valores, obteniendo:

$$T(n) = \Theta(1 \cdot \log\ n\ ) = \Theta(\log\ n\ ).$$

2) **Dado un arreglo N-1 enteros ordenados, cuyos valores están en el rango 1 a N, encontrar el entero faltante dado que uno de ellos no está presente en el arreglo**

N= [1, 2, 3, 4, 6, 7, 8]; N = [1, 2, 3, 4, 5, 6, 8, 9]

Para este ejercicio al igual que en el anterior se escoge la búsqueda binaria, ya que el arreglo esta ordenado, permitiendo saber que numero es el que falta debido a este orden, identificando el número deseado. Esto permite aplicar la búsqueda binaria y evita recorrer todo el arreglo, reduciendo el espacio de búsqueda.

Complejidad

Línea	Costo	Tiempo
low=0	c_1	1
high=len(A)	c_2	1
while low<high:	c_3	$\log(n+1)$
mid=low+(high-low)//2	c_4	$\log n$
if A[mid]==k:	c_5	$\log n$
return mid	c_6	$1;\ A[mid] == k$
else:	c_7	1
if a[mid]>k:	c_8	$\log n$
High=mid	c_9	$\log n\,;\ A[mid] > k$
Else:	c_{10}	1
Low=mid+1	c_{11}	$\log n$

Teorema Maestro

$$T(n) = aT(n/b) + f(n)$$

Donde

$$a = 1;\ b = 2;\ f(n) = \Theta(1)$$

Calculo de n^log_b a

$$n^{\log_b} a = n^{\log_2} 1 = n^0 = 1$$

Resultado

Según el caso 2 cuando $f(n) = \Theta\left(n^{\log_b} a\right)$, entonces $T(n) = \Theta\left(n^{\log_b} a \log n\right)$ y sustituimos los valores, obteniendo:

$$T(n) = \Theta(1 \cdot \log\ n\ ) = \Theta(\log\ n\ ).$$

3) **Diseñar un algoritmo D&Q para calcular el exponente de un numero a^n.**

La idea principal del programa es partir el exponente a la mitad ya que por propiedades de los exponentes: $a^n = a^{\frac{n}{2}} + a^{\frac{n}{2}}$ y de esta forma sucesivamente. Sin embargo, para poder hacer esto se debe dividir el problema en pares e impares, siendo los pares a^$n = a^{\frac{n}{2}} + a^{\frac{n}{2}}$ y los impares $a^n = a * (a^{\frac{n}{2}} + a^{\frac{n}{2}})$.

Código

```
1   def potencia(a, n):
2       if n == 0:
3           return 1
4       temp = potencia(a, n//2)
5       if n % 2 == 0:
6           return temp * temp
7       else:
8           return a * temp * temp
```

Complejidad

Línea	Costo	Tiempo
If n==0	c_1	1
Return 1	c_2	1
Temp=potencia(a,n//2)	$c_3 + T\left(\frac{n}{2}\right)$	1
If n%2==0	c_4	1
Return temp*temp	c_5	1
Else:	c_6	1
Return a*temp*temp	c_7	1

Teorema Maestro

$$T(n) = aT(n/b) + f(n)$$

Donde

$$a = 1;\ b = 2;\ f(n) = \Theta(1)$$

Calculo de n^log_b a

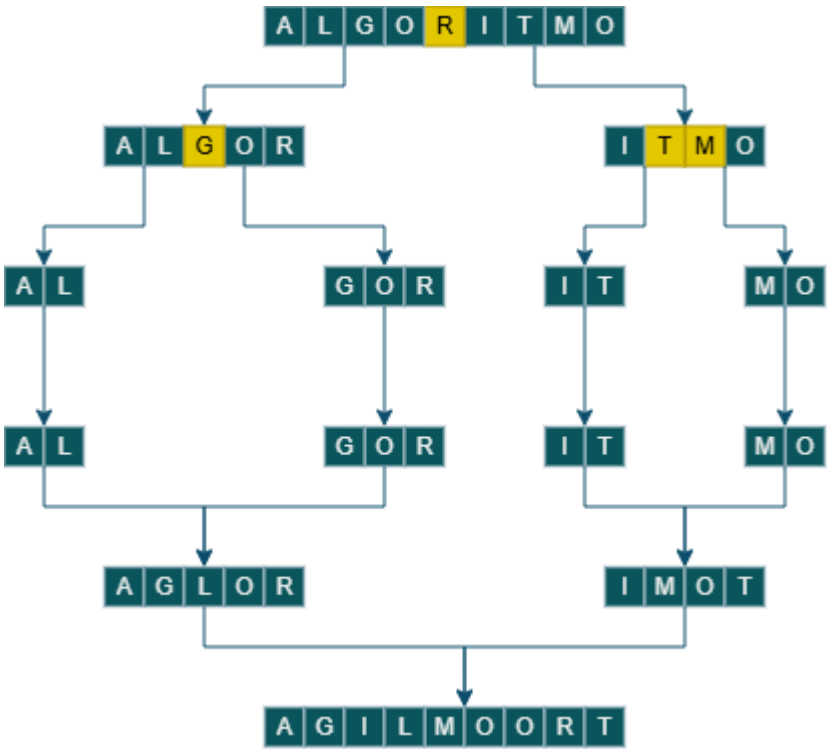
$$n^{\log_b} a = n^{\log_2} 1 = n^0 = 1$$

Resultado

Según el caso 2 cuando $f(n) = \Theta\left(n^{\log_b} a\right)$, entonces $T(n) = \Theta\left(n^{\log_b} a \log n\right)$ y sustituimos los valores, obteniendo:

$$T(n) = \Theta(1 \cdot \log\ n\ ) = \Theta(\log\ n\ ).$$

4) Aplicar mergesort para ordenar "ALGORITMO" en orden alfabético.



Complejidad

Línea	Costo	Tiempo
If len(A)<=1	c_1	1
Return A	c_2	1
mid=len(A)//2	c_3	1
Left=merge_sort(A[:mid])	$c_4 + T\left(\frac{n}{2}\right)$	1
right=merge_sort(A[mid:])	$c_5 + T\left(\frac{n}{2}\right)$	1
Return merge(left,right)	c_6	$\Theta(n)$

Teorema Maestro

$$T(n) = aT(n/b) + f(n)$$

Donde

$$a = 2;\ b = 2;\ f(n) = \Theta(1)$$

Calculo de n^log_b a

$$n^{\log_b} a = n^{\log_2} 2 = n^1 = n$$

Resultado

Según el caso 2 cuando $f(n) = \Theta\left(n^{\log_b} a\right)$, entonces $T(n) = \Theta\left(n^{\log_b} a \log n\right)$ y sustituimos los valores, obteniendo:

$$T(n) = \Theta(n \log n).$$

5) **Dado un numero X (X=5), encontrar un numero N tal que la suma de los bits de cada número desde 1 hasta N sea al menos X.**

En otras palabras, la sumatoria de los bits desde 1 hasta N debe ser ≥ X.

n	binario	Bits
1	1	1
2	10	1
3	11	2
4	100	1
5	101	2

Suma acumulada de bits

$$1 \rightarrow 1 + 1 = 2 \rightarrow 2 + 2 = 4 \rightarrow 4 + 1 = 5$$

Esto significa que en la posición n=4 la sumatoria de bits da 5, cumpliendo con la condición.